SB5-MAI Software Maintenance — Course Overview

> Course map, the software-change process, the JHotDraw case study, the lecture index, and the bibliography.

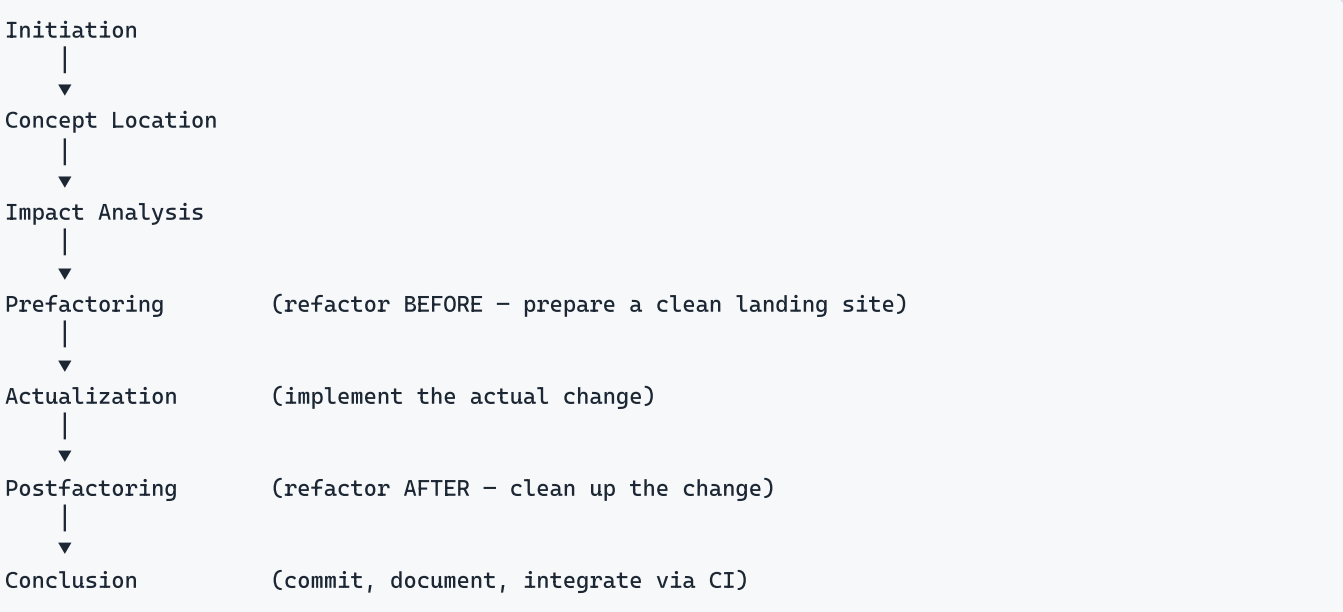
The Software-Change Process

Software maintenance in this course is framed not as ad-hoc patching but as a disciplined, repeatable *software-change process* applied to existing code. Software change is defined as *the process of adding new functionality to existing code* — the foundation of software evolution and servicing — and it always operates on a codebase that already runs, which is what distinguishes maintenance from green-field development. Following Rajlich, every change to a real system is carried out as an ordered sequence of eight phases. The spine runs from understanding the request, through locating and analysing the code that must change, restructuring so the change fits, making the change itself, restructuring again to clean up, and finally closing out and verifying. The phases group into three roles: *interactions with the world* (Initiation, Conclusion), *change-design* (Concept Location, Impact Analysis), and *change-implementation* (Prefactoring, Actualization, Postfactoring); Verification runs vertically alongside the implementation phases rather than as a single final step. Each phase consumes a definite input and produces a definite hand-off that the next phase depends on, so the chain is a pipeline, not a checklist. The eight phases, in order:

1. **Initiation** — *What it is:* the entry phase, where a change request first arrives from a stakeholder (a user reporting a bug, a user or manager asking for an enhancement, or a programmer proposing an improvement). *What it is for:* to capture the request as a clear requirement, prioritise it in the Product Backlog (Must / Nice-to-have / Won't have), and accept it into the process so its scope and goal are pinned down before any code is touched. *It produces:* a well-formed change request — ideally a User Story ("As a [user type], I want [goal] so that [reason]") small enough to fit a 3×5 card — that becomes the starting point for Concept Location. *Example:* a developer rewrites a vague "add watermarks to exports" email into a single user story and files it as a prioritised backlog card.
2. **Concept Location** — *What it is:* the activity of finding the exact code snippet where a change must begin. *What it is for:* to bridge the vocabulary gap between the request (written in domain concepts) and the code (where those concepts are scattered and renamed), so the developer reaches the minimum essential understanding via an as-needed strategy rather than reading the whole system. *It produces:* the first class/method that realises the named concept — the "location" — which seeds the **initial impact set** handed to Impact Analysis. *Techniques:* human knowledge, traceability tools, dynamic search (execution traces / IDE debugger), and static search (dependency search over a Class Dependency Graph, GREP pattern matching, information retrieval). *Example:* a developer extracts the concepts *Export, Drawing, Text, Format* from the request, then runs dependency search from the export controller class until a class that *locally* implements export is found.
3. **Impact Analysis** — *What it is:* the phase that grows the single located concept into the complete set of components the change will touch. *What it is for:* to determine the change strategy and bound its reach before implementation, so cost and risk are understood up front. *It produces:* a stabilised **impact set** — starting from the concept-location classes, each is marked *impacted* and the mark propagates through dependencies until no new classes are added; tools such as JRipples support this incremental marking. *Example:* a developer marks the export class impacted, then walks its callers and the format-writer classes, adding each to the impact set until it stops growing.

4. **Prefactoring** — *What it is:* opportunistic, behaviour-preserving refactoring performed *before* the new behaviour is added. *What it is for:* to localise and minimise the impact of the upcoming change — to prepare a clean "landing site" free of duplication and code smells so the change drops in cleanly. *It produces:* a restructured-but-equivalent codebase (tests stay green) ready to receive new code, using refactorings such as Extract Class or Extract Superclass. *Example:* a developer pulls export-formatting logic into its own class so the watermark step has one obvious place to live.
5. **Actualization** — *What it is:* the phase where the actual new or changed behaviour is written. *What it is for:* to create the new code, plug it into the existing code, and propagate the change to neighbouring classes so the system stays consistent (managing change propagation and the ripple effect). *It produces:* a working system that now exhibits the requested functionality, integrated with the prefactored structure. *Example:* a developer adds the watermark-stamping code and wires it into each export path so every format carries the watermark.
6. **Postfactoring** — *What it is:* behaviour-preserving refactoring performed *after* the change, the mirror image of Prefactoring. *What it is for:* to remove any anti-patterns the new code introduced (e.g. a now-overlong method or a bloated class), restoring clean structure and design integrity. *It produces:* a cleaned-up codebase equivalent in behaviour to the just-actualized one but better-structured for the next change. *Example:* a developer splits the export method that grew too long during Actualization back into focused methods.
7. **Conclusion** — *What it is:* the closing world-interaction phase. *What it is for:* to commit the finished code to version control, build the new baseline, update documentation, integrate via CI, and close the change request so the system returns to a stable, releasable state ready for the next change. *It produces:* a released baseline and a closed backlog item, looping the process back toward Initiation. *Example:* a developer merges the feature branch, the CI pipeline builds and tags the new baseline, and the backlog card is marked done.
8. **Verification** — *What it is:* the correctness guarantee that runs continuously across the implementation phases (Prefactoring → Conclusion), not a single final gate. *What it is for:* to confirm the change is correct and complete — tests pass (functional, unit, structural), acceptance criteria such as BDD scenarios are met, walkthroughs are clean, and no regressions were introduced. *It produces:* the evidence that the changed system behaves as required, justifying the Conclusion. *Example:* a developer runs the JUnit suite and a JGiven acceptance scenario asserting the watermark appears in every export format before allowing the commit.

Phase diagram (process spine):



```
    |
    ▼
Verification       (tests / BDD acceptance — confirm correctness)
```

Prefactoring and Postfactoring bracket Actualization: clean the site, make the change, clean up after. Verification confirms the result.

The JHotDraw Case Study

JHotDraw is an open-source Java GUI framework for building structured drawing editors — applications where the user places boxes, lines, connectors, text, and other *figures* on a canvas and manipulates them with *tools* and *handles* (resize/move grips). It began life as **HotDraw**, written in Smalltalk by Kent Beck and Ward Cunningham, and is remembered as one of the first software projects explicitly designed for reuse and labelled a "framework"; the Java port is JHotDraw, with JHotDraw 7 (Randelshofer's documentation and handbook) referenced in the course readings. Architecturally it is a Model-View-Controller framework whose core classes — `DrawApplication`, `StandardDrawingView`, `Drawing`, `Figure`, `Tool`, and `Handle` — are deliberately built on Gang-of-Four design patterns (Composite for figures-within-figures, Strategy for layouters, State for tool modes, Template Method for line connections, Decorator for borders/shadows, Factory Method for menus/tools, Prototype for cloning figures). It is widely studied as an exemplar of clean, pattern-rich, maintainable Java code.

A drawing framework is an excellent teaching vehicle for maintenance precisely because it was *designed for change*: its responsibilities are well separated and its extension points are explicit (you subclass `Tool` and `Figure` to add your own), so the textbook moves — concept location, impact analysis, prefactoring — can be taught cleanly on a real codebase instead of a toy. It is large and real enough to be non-trivial, yet small, well-named, and well-documented enough to comprehend in a single semester; its pattern-aligned classes map cleanly onto domain concepts, which is exactly what makes locating a concept tractable. Across the course students *use* JHotDraw as the running target for every phase: they fork it on GitHub, write **user stories** against existing features (L02 lab), run **concept location** and build an initial class set with the IDE debugger, perform **impact analysis** against its actual class graph, **refactor** its code (pre- and post-factoring), study and extend its **design patterns**, and add features end-to-end — making the framework a shared, hands-on portfolio playground for practising maintenance on existing software. The supporting literature — Kaiser's "Become a programming Picasso with JHotDraw", Kirk's "JHotDraw Pattern Language", Savolskyte's and Pavlos's reviews/specifications, and Randelshofer's JHotDraw 7 documentation — supplies the shared reference material. The related **Drawlets** drawing framework appears in the L10 worked example as a parallel case for applying the full process end-to-end.

Lecture Index

Lecture	Title	Primary topics	Process phase(s)
L01	Introduction & Version Control	Course intro, software maintenance overview, Git/GitHub workflow	(process overview)
L02	Software Change & Concept Location (introduces JHotDraw)	The software-change model, concept location, JHotDraw case study	Initiation, Concept Location
L03	Impact Analysis, Software Processes & CI	Impact sets, change propagation (JRipples), software processes, continuous integration	Impact Analysis, Conclusion

Lecture	Title	Primary topics	Process phase(s)
L04	Refactoring & Maintainable Code	Refactoring catalog, code smells, maintainable-code guidelines	Prefactoring, Postfactoring
L05	Actualization, Clean Architecture & OO Principles	Implementing the change, clean architecture, SOLID/OO principles	Actualization
L06	Clean Code & Design Patterns	Clean code practices, GoF design patterns, refactoring to patterns	Prefactoring, Postfactoring, Actualization
L07	Software Testing	Unit testing (JUnit/AssertJ), xUnit patterns, test doubles, test automation	Verification
L09	BDD & Verification	Behaviour-Driven Development, acceptance criteria (JGiven), verification	Verification
L10	Conclusion & Worked Example (Drawlets)	Closing the change, end-to-end worked example on Drawlets	Conclusion, Verification (full spine)
L11	Technical Debt	Technical debt: definition, interest/principal, management strategies	(cross-cutting / Conclusion)

(Lectures 8 and 12 have no material and are omitted.)

Key Themes Across the Course

- **Maintainability** — *What it is:* the degree to which software can be understood, located within, and safely changed over its whole lifetime. *Why it matters:* maintenance (mostly *perfective* — adding and improving functionality) dominates the real cost of software, so code that is cheap to evolve is the whole point of the discipline. The "Building Maintainable Software" ten guidelines (`[VRvdL+16]`) and the change process itself exist to keep code economical to change again and again, not just to ship once.
- **Refactoring** — *What it is:* changing a program's internal structure *without changing its observable behaviour*. *Why it matters:* it is the connective tissue of the change process, appearing explicitly as *Prefactoring* (before the change, to minimise its impact) and *Postfactoring* (after the change, to remove introduced anti-patterns). Behaviour-preservation is what lets developers improve design continuously without breaking working software; backed by Fowler's refactoring catalog (`[Fow11]`) and Kerievsky's *Refactoring to Patterns* (`[Ker05]`).
- **Testing & Verification** — *What it is:* the practices that demonstrate a change is correct — unit/functional/structural testing plus walkthroughs and acceptance scenarios. *Why it matters:* without a safety net, neither refactoring nor actualization can be done with confidence; tests are what make change *safe* and let Verification certify the result. The course covers unit testing (JUnit, AssertJ, xUnit patterns) and rises to Behaviour-Driven Development (`[Nor06]` , JGiven) so that *Verification* confirms the change against the original acceptance criteria.
- **Clean Code & Clean Architecture** — *What it is:* day-to-day craftsmanship at the statement/method level (`[MC09]` *Clean Code*) plus structural discipline at the module level (clean architecture, SOLID and other OO principles). *Why it matters:* comprehensible, well-structured code is what keeps the early phases cheap — concept location and impact analysis stay fast and accurate when names are clear and dependencies are clean, so maintenance does not degrade over time.
- **Design Patterns** — *What it is:* named, reusable solutions to recurring OO design problems — the 23 Gang-of-Four patterns (`[GHJV94]`) in creational, structural, and behavioural families. *Why it matters:* patterns are the shared vocabulary for the designs students read, refactor, and extend, and they are the

change-enablers that let frameworks absorb new requirements with minimal impact — JHotDraw being the canonical pattern-rich example where each pattern maps to a low-impact extension point.

- **Technical Debt** — *What it is:* the implied future cost of expedient shortcuts, framed as a loan with *principal* (the cost to do it right) and *interest* (the extra effort every future change pays while it stands). *Why it matters:* it is the economic lens that ties the course together — unmanaged debt compounds until the system becomes hard to change, and the change process plus refactoring discipline are precisely the mechanisms for paying it down before it does.

Bibliography & Citation Keys

Keys and references below are taken verbatim from the course **Literature List** PDF (`Lecture 1/[Litt] Literature List.pdf`). The course's own keys are used as the canonical citation keys. Two convenience aliases requested by the study package are noted where they apply.

Key	Full reference
`[Raj13]`	Václav Rajlich. *Software Engineering: The Current Practice*, volume 38. ACM, New York, NY, USA, November 2013. **(course textbook)** — *The book that defines the eight-phase software-change process at the heart of this course.*
`[GHJV94]`	Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software* ("Gang of Four"). Addison-Wesley Professional, 1st ed., November 1994. — *The catalog of 23 reusable OO design patterns (creational, structural, behavioural) that JHotDraw is built on.*
`[GOF96]`	*Alias for* `[GHJV94]`. The Gang-of-Four *Design Patterns* book as cited inside Robert C. Martin's *Design Principles and Design Patterns* deck (some printings date it 1995/1996). Normalize to `[GHJV94]`.
`[OCP97]`	Robert C. Martin. *The Open-Closed Principle*. C++ Report / "Engineering Notebook" column, SIGS Publications, 1996. (OCP)
`[LSP97]`	Robert C. Martin. *The Liskov Substitution Principle*. C++ Report / "Engineering Notebook" column, SIGS Publications, 1996. (LSP)
`[DIP97]`	Robert C. Martin. *The Dependency Inversion Principle*. C++ Report / "Engineering Notebook" column, SIGS Publications, 1996. (DIP)
`[ISP97]`	Robert C. Martin. *The Interface Segregation Principle*. C++ Report / "Engineering Notebook" column, SIGS Publications, 1996. (ISP)
`[Granularity97]`	Robert C. Martin. *Granularity* (the package-cohesion principles: REP, CRP, CCP). C++ Report / "Engineering Notebook" column, SIGS Publications, 1996–97.
`[Stability97]`	Robert C. Martin. *Stability* (the package-coupling principles: ADP, SDP, SAP). C++ Report / "Engineering Notebook" column, SIGS Publications, 1996–97.
`[Liskov88]`	Barbara Liskov. *Data Abstraction and Hierarchy*. OOPSLA '87 Addendum / SIGPLAN Notices 23(5), May 1988. (Origin of the Liskov Substitution Principle.)
`[Martin99]`	Robert C. Martin. *Design Principles and Design Patterns*. ObjectMentor, 1999–2000. (The article behind the L06 `DesignPrinciplesAndPatterns` deck; collects the SOLID and package principles.)
`[MC09]`	Robert C. Martin and James O. Coplien. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, Upper Saddle River, NJ, 2009. — *The course's source for day-to-day code craftsmanship (naming, functions, comments, formatting).*
`[VRvdL+16]`	Joost Visser, Sylvan Rigal, Rob van der Leek, Pascal van Eck, Gijs Wijnholds. *Building Maintainable Software, Java Edition: Ten Guidelines for Future-Proof Code*. O'Reilly Media, 1st ed., 2016. — *The ten concrete, measurable guidelines for writing future-proof, maintainable code.*

Key	Full reference
`[Ker05]`	Joshua Kerievsky. *Refactoring to Patterns*. Addison-Wesley, 2005. — *Marries Fowler-style refactoring with GoF patterns: refactoring toward (and away from) patterns as the design demands.*
`[Larman04]`	Craig Larman. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall, 3rd ed., 2004. (Source of the GRASP responsibility-assignment patterns.)
`[Bec99]`	Kent Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 1999.
`[Fow06]`	Martin Fowler. *Continuous Integration*, 2006. (HTML)
`[Fow11]`	Martin Fowler. *Refactorings in Alphabetical Order*, 2011. (HTML) — *The canonical catalog of named refactorings (Extract Class, Extract Superclass, …) used in pre- and post-factoring.*
`[Nor06]`	Dan North. *Introducing BDD*, 2006. (HTML) — *The origin article for Behaviour-Driven Development and Given/When/Then acceptance scenarios used in Verification.*
`[BBPR05]`	Jonathan Buckner, Joseph Buchta, Maksym Petrenko, Václav Rajlich. *JRipples: A Tool for Program Comprehension during Incremental Change*. IWPC, pp. 1–4, 2005. — *Describes JRipples, the tool that supports incremental impact-set marking during Impact Analysis.*
`[BMZ+05]`	Jim Buckley, Tom Mens, Matthias Zenger, Awais Rashid, Günter Kniesel. *Towards a Taxonomy of Software Change*. Journal of Software Maintenance and Evolution, 17(5):309–332, September 2005.
`[RG04]`	V. Rajlich, P. Gosavi. *Incremental Change in Object-oriented Programming*. IEEE Software, 2004.
`[Cos16]`	Joel Costigliola. *AssertJ*, 2016. (HTML)
`[JUn]`	JUnit. *JUnit Cookbook*. (HTML)
`[Mes03]`	Gerard Meszaros. *The Test Automation Manifesto*, 2003. (PDF)
`[Mes08]`	Gerard Meszaros. *xUnit Patterns*, 2008. (HTML)
`[Sch16]`	Jan Schäfer. *JGiven*, 2016. (HTML)
`[Sou]`	SourceMaking.com. *Refactorings*. (HTML)
`[Lea16]`	LearnCode. *Github Tutorial*, 2016. (Video)
`[Kai01]`	Wolfram Kaiser. *Become a Programming Picasso with JHotDraw*, 2001. (HTML) — *JHotDraw — A practical introduction to building drawing editors with JHotDraw and its patterns.*
`[Kir01]`	Douglas Kirk. *JHotDraw Pattern Language*, 2001. (HTML) — *JHotDraw*
`[Sav01]`	Jolita Savolskyte. *Review of the JHotDraw Framework*, 2001. (PDF) — *JHotDraw*
`[Pav11]`	Nikolaidis Pavlos. *Software Requirements Specification for JHotDraw*, 2011. (PDF) — *JHotDraw*
`[Ran11a]`	Werner Randelshofer. *JHotDraw 7 Documentation*, 2011. (HTML) — *JHotDraw*

Key	Full reference
`[Ran11b]`	Werner Randelshofer. *The JHotDraw 7 Handbook*, 2011. (PDF) — *JHotDraw*
`[Ols12a]`	Andrzej Olszak. *Featureous: an integrated approach to location, analysis and modularization of features in java applications*. PhD thesis, 2012. (PDF)
`[Ols12b]`	Andrzej Olszak. *Introduction to Featureous*, 2012. (Video)
`[Sør15a]`	Jan Sørensen. *Introduction to Software Maintenance*, 2015. (Video)
`[Sør15b]`	Jan Sørensen. *Overview of Software Maintenance Syllabus*, 2015. (Video)
`[Sør15c]`	Jan Sørensen. *Review Questions — Introduction to Software Maintenance*, 2015. (HTML)

Alias notes (best-effort, for the study package): - `[Fowler99]` — the study package sometimes refers generically to Fowler's *Refactoring*. The course Literature List contains **no** Fowler 1999 book entry; the canonical Fowler refactoring source here is `[Fow11]` (*Refactorings in Alphabetical Order*). Treat `[Fowler99]` as an alias for `[Fow11]`. - `[Martin]` — generic alias for Robert C. Martin's *Clean Code* / *Clean Architecture* writings. In this Literature List the concrete entry is `[MC09]` (*Clean Code*, Martin & Coplien). Clean Architecture content is course-supplied and not a separate list entry; cite `[MC09]` for Clean Code and label Clean Architecture material as deck-sourced.

How to Use This Study Package

This package has three complementary parts; use them together:

1. **Guides (this corpus)** — Read `00-overview.md` first to internalise the eight-phase change process and the JHotDraw case, then work through `lecture-01-*.md … lecture-11-*.md`. Each guide is fully cited; every reading reference uses the citation keys defined above. The H2 sections are self-contained, so you can jump straight to a single concept.
2. **Lookup search** — Use the offline semantic search (Lookup mode) to ask natural-language questions ("how does impact analysis propagate?", "what is prefactoring?") and retrieve the most relevant guide sections. Filter by **topic** (the controlled vocabulary in `data/mcqs/_taxonomy.md`) or by lecture to narrow results.
3. **Quiz practice** — After reading, drill with Quiz mode. MCQs are tagged by the same controlled **topic** vocabulary, by **lecture**, and by **difficulty** (`easy | medium | hard | very-hard`). Filter to a weak topic, answer, then read the explanation (which defeats every distractor and ends in a citation) to close the loop back to the guides.

Recommended loop: **read the guide → search to clarify gaps → quiz the topic → revisit the cited guide section on any miss.**